

AI 时代的编程指南

深入系统



Charles Tao 著，与 Claude 协作完成

books.stickmancharles.com · v1.0

正文 CC BY-NC-ND 4.0 · 代码 MIT

署名: Charles Tao, 《AI 时代的编程指南》, books.stickmancharles.com

AI 时代的编程指南 · 第四册《深入系统》

下车，打开引擎盖——四大件从"会用"到"懂它"。

作者：Charles Tao（与 Claude 协作完成） 版本：v1.0 · 2026 年 7 月 · 系列共五册，本册难度
★★★★

小序

前三册教你在这个世界里开车：认地图、握方向盘、读路牌。你已经能用四大件（Cloudflare + Vercel + Supabase + DO）做出真东西了。但"会用"和"懂它"之间隔着一层薄板，平时感觉不到——出事的那天，这层薄板就是天堑。这一册把板子掀开。

十章的路线：先逐个打开四大件里最陌生的两件（Cloudflare 这位门口的保安、DO 那台裸服务器），再补上把一切连起来的隐形零件（进程与服务、端口与代理），然后深入数据这一层（表设计、RLS 权限），最后是三门"懂系统的人才有的"功课：API 设计、性能与缓存、日志监控，以及——读账单。

本册有个贯穿装置：每章末尾多一条"出事时想起我"，列出这一层出毛病时的典型症状。十章读完，你会攒出一张"症状 → 层"的对照表。这张表是工程师身上最值钱的直觉：别人看到的是"网站坏了"，你看到的是"这个症状属于第几层，该翻哪本日志"。

最后一件事说在前头：本册的试一试，第一次真正触碰生产环境。所以规矩升级——每个试一试都标了安全档位，默认从"只读"开始；标 △ 的，先在测试环境走熟，或者和爸爸一起做。开读之前，把第一册附录 C 的红线卡找出来，放在手边。引擎盖下面没有魔法，但有高压电。

目录

- Cloudflare 全解：门口那位保安到底在干什么
- 登上我们的 DO：SSH、密钥、在一台裸服务器上认路
- 让程序永远醒着：进程、后台服务、开机自启
- 端口与反向代理：为什么是 443，Nginx 在门口排班
- 把表设计好：关系、外键、索引

6. RLS 策略实战：亲手写"谁能看哪一行"
7. 自己开窗口：设计一个像样的 API
8. 缓存与性能：快，是设计出来的
9. 日志、监控与告警：出事之前先知道
10. 云账单怎么读：成本也是架构的一部分

附：本册新词

第 1 章 Cloudflare 全解：门口那位保安到底在干什么

第一册第 6 章介绍过：我们的域名管在 Cloudflare 手里，它既当电话簿，又兼任调度员和保安。这一章把三个身份逐个拆开——从此它面板上的每个开关，你都知道背后是什么。

身份一：电话簿（DNS 托管）。打开 Cloudflare 的 DNS 页面，那张记录表现在可以逐列读了：Type（记录类型）、Name（名字，@ 代表根域名本身）、Content（指向哪）、TTL（缓存多久）。几条真实形状的记录——CNAME | www | cname.vercel-dns.com：www 指向 Vercel；TXT | @ | v=spf1...：邮件相关的“便签”；A | api | （一串 IP）：api 子域名直指某台机器，比如我们的 DO。都是第一册的老朋友，只是第一次看到它们住在真实的表里。

但表里还有一列是第一册没讲的，也是 Cloudflare 最重要的开关：那朵云。橙色（Proxied）= 代理开启：访客的流量先进 Cloudflare 的机房，再转给你；灰色（DNS only）= 只做电话簿，访客直连目标。橙云一开，两件大事同时发生：其一，Cloudflare 成了你的门面，后面真正干活的机器——术语叫源站（Origin）——的 IP 被藏了起来，坏人看不见真门牌；其二，下面两个身份才得以生效。

身份二：调度员（CDN 与缓存）。橙云开着时，静态文件（图片、CSS、JS）会被缓存在全球各地的节点，访客就近取货——第一册第 5 章的 CDN，就是它在干。缓存意味着“答案可能是旧的”：所以面板上有 Purge Cache（清空缓存）按钮——“我明明改了，页面还是旧的”这个千古谜题，一半的答案在这里（另一半在第 8 章）。

身份三：保安（防护）。挡人海攻击（DDoS：用海量假流量把你淹死，Cloudflare 用它更大的身板替你扛）、拦恶意爬虫、对可疑访客弹“验证你是人类”的质询页——你在别人网站上见过的那一页，就是这位保安在上班。顺带一条重要设置：Cloudflare 到源站这一段也要加密（SSL 模式选 Full 或 Strict），别用 Flexible——那等于前半程加密、后半程裸奔。

出事时想起我

一部分人打得开、一部分人打不开，或改了记录不生效——DNS 传播加缓存，等，或 Purge；网站全挂但 Vercel 后台显示一切正常——查 Cloudflare 状态和 DNS 配置，门面倒了后厨再好也没用；浏览器报“重定向次数过多”——十有八九是 SSL 模式配错的经典症状。

关键领悟

Cloudflare 的一切能力都源于一个事实——橙云一开，所有流量先过它的手。理解这一点，缓存、防护、藏源站，全是自然推论。

试一试（安全档：只读）

打开 Cloudflare 面板，把我们域名的每条记录读一遍：类型、指向、橙云还是灰云，以及"它为什么存在"。有一条答不上来的，拿去问 AI——每条记录都该有存在的理由，答不上来的那条最值钱。

第 2 章 登上我们的 DO：SSH、密钥、在一台裸服务器上认路

前三册里，你一直隔着平台的玻璃看服务器：Vercel 替你部署，Supabase 替你托管。今天不隔了——亲自登上我们在 DigitalOcean 租的那台机器。

登船工具叫 SSH（Secure Shell）：一条加密的远程终端。你在自己电脑的终端里敲命令，命令在地球另一端那台机器上执行，结果传回你眼前。第二册练的终端功夫，今天在远方兑现。

先说钥匙。SSH 登录用的是密钥对：公钥像挂在服务器门上的锁，可以随便给人看；私钥是你兜里的钥匙。私钥的纪律，是第一册第 9 章秘密纪律的最高级形态：永不离开你的电脑，永不进 Git，永不发给任何人——包括 AI。让 AI 帮你排查 SSH 问题时，它需要看的是报错信息，永远不是你的私钥文件。这条写进 CLAUDE.md 也不为过。

登录本身一行：`ssh root@服务器IP`。第一次连接会弹出一段指纹（fingerprint）确认——那是在记住这台机器的“长相”，防止有人中途冒充，确认一次，以后它就认得了。

然后是登船后最惊喜的部分：你全都会。`pwd` 看看自己在哪（root 用户的家是 `/root`）；`ls` 逛逛；`df -h` 看磁盘还剩多少；`free -h` 看内存；`top` 看现在谁在干活、谁在吃 CPU（按 q 退出）。每个命令都是第二册第 4 章的老朋友，只是换了台机器执行。这一刻值得停下来体会：服务器不神秘，它就是一台没有屏幕、永远开机、拿 SSH 当显示器的 Linux 电脑。

但有一件事和你的笔记本不同：你现在是 root——这台机器的神。没有回收站，没有确认框，`rm` 说删就删。第一册红线卡在这台机器上字字生效，而且加倍：这里跑的是生产环境。（正经做法是建一个普通用户、需要时用 `sudo` 提权，让“当神”成为显式动作——这件事可以让 AI 带你配一次。）

最后合个龙：ForgeCard 那台树莓派，你们平时是怎么进去改代码的？也是 SSH。原来这门功夫你们早就会了——DO 只是把树莓派搬进了别人的机房，再接上永不断的电和网。本册后面几章，树莓派都是绝佳的练手机：同样的系统，砸了也不心疼。

出事时想起我

SSH 连不上——先分清是机器挂了还是网络不通（ping 一下），再想防火墙；服务器莫名罢工、服务写不进数据——`df -h` 查磁盘，磁盘被日志撑满是裸服务器的头号隐形杀手；机器卡成蜗牛——`top` 里找那个吃满 CPU 或内存的进程。

关键领悟

SSH 之后没有新知识，只有旧知识的新舞台。真正的新东西是分量——root 加生产环境，等于每条命令都是实弹。

试一试（安全档：只读；建议先拿树莓派练）

SSH 登上去，只跑五个只读命令：`pwd`、`ls`、`df -h`、`free -h`、`top`。看一圈，`exit` 下船。什么都不改——今天只是认门。

第 3 章 让程序永远醒着：进程、后台服务、开机自启

一个所有人都会撞上的谜题：你 SSH 上去，手动跑 `python3 read_card.py`，程序欢快地转起来；你断开 SSH——它死了。为什么？因为你手动启动的程序是你这次会话的孩子，会话结束，孩子被一并收走。而我们要的，是第一册第 5 章承诺过的那种"永远醒着"。

先补一个正名：进程（Process）——正在运行中的程序。程序是躺在磁盘上的文件，进程是它活过来的样子，每个进程有个编号（PID）。`top` 里滚动的那一列就是它们；`kill PID` 是客气地请某个进程退出（`kill -9` 是强制驱逐，最后手段）。

让程序脱离你的会话、永远活着，正规军方案叫 `systemd`——Linux 自带的"服务管家"。思路是登记制：写一张登记表（叫 unit 文件），告诉管家这个程序是谁、怎么启动、死了要不要扶、开机要不要自动跑。一张最小的登记表长这样：

01ini

```
[Unit]
Description=ForgeCard reader

[Service]
ExecStart=/usr/bin/python3 /home/pi/forgecard/read_card.py
Restart=always

[Install]
WantedBy=multi-user.target
```

逐行读：它叫什么；用哪条命令启动；`Restart=always`——死了自动扶起来；最后一行大意是"正常开机流程里带上它"。登记之后，指挥管家只需四个命令：`systemctl start/stop/restart/status 名字`（起、停、重启、看状态），外加 `systemctl enable 名字`（开机自启）。还有一条马上就离不开的：`journalctl -u 名字 -f`——看这个服务的日志，`-f` 是第二册学过的"追着看"。

这里有一条值得单独说破的哲学：`Restart=always` 承认了一个真相——程序会崩。与其追求写出永不崩溃的程序（没人做得到，AI 也不行），不如让管家把"崩了就扶起来"变成系统的本

能。这和我们整套书的心法一脉相承：不指望不犯错，指望错误发生时有人接住。

合龙时刻：ForgeCard 的树莓派一通电就自动开始读卡，等的就是这套机制（或它的同类）。你们早就在享受"服务管家"的照顾，今天终于知道了它的名字。顺带认个脸：Node 世界常见一个叫 pm2 的工具，同样的思路、另一位管家，见到不慌。

出事时想起我

"SSH 一断程序就死"——没登记给管家，还在裸跑；"重启服务器后服务没起来"——登记了但没 `enable`；"服务显示在跑却不干活"——`systemctl status` 看状态，`journalctl -u` 读日记，答案几乎总在里面。

关键领悟

"永远醒着"从来不是程序自己的本事，是管家的本事。把程序登记给 `systemd`，活着就成了系统的责任，而不是你的运气。

试一试（安全档：只读，树莓派上做）

`systemctl status` 加服务名，看看读卡服务的登记信息和当前状态；再 `journalctl -u 服务名 -n 20` 读它最近二十行日记。想练手写登记表的话，让 AI 陪你在树莓派上给一个无害的小脚本（比如每分钟写一行时间戳）注册成服务，走完 `start`、`status`、`enable`、重启验证的全流程。

第 4 章 端口与反向代理：为什么是 443，Nginx 在门口排班

一台机器一个 IP，可上面同时跑着好几个程序——访客的请求进了楼，该敲哪扇门？答案是端口（Port）：IP 找到楼，端口找到房间。房间号从 0 到 65535，其中三个名门必须背下：80 = HTTP，443 = HTTPS，22 = SSH。浏览器里输网址不用写端口，是因为它默认拨 443；而你本地开发时那个 `localhost:3000`，此刻也彻底通了——你的开发服务器住在自己电脑的 3000 号房。

于是新问题来了：DO 上跑着一个监听 3000 的程序，总不能让全世界访问 `我们的域名:3000` 吧？而且 443 的门口需要 HTTPS 证书，谁来管？答案是在大门口安排一位前台：反向代理（Reverse Proxy），最常见的这位叫 Nginx。所有访客一律敲 443 的大门，Nginx 接待，按规则把请求转给屋里正确的房间，再把回答递出去。它的排班表长这样：

📄 nginx

```
server {  
    listen 443 ssl;  
    server_name api.example.com;  
  
    location / {  
        proxy_pass http://localhost:3000;  
    }  
}
```

逐行读：听 443 号门（带加密）；只认这个域名的访客；一切请求，转给本机 3000 号房。为什么叫“反向”？一句话：正向代理替访客出门办事，反向代理替服务器在门口接客。Nginx 顺手还干几件事：挂 HTTPS 证书（配合 Let's Encrypt 这类免费证书自动续期）；一台机器伺候多个域名（不同 `server_name` 分流到不同房间）；静态文件不劳烦后面的程序、自己直接发。

现在可以做一次全景合龙了。把第一册第 1 章那“半秒钟”补成完整旅程：浏览器 → Cloudflare（橙云，第 1 章）→ 源站 443 号门 → Nginx 前台（本章）→ `localhost:3000` 的进程（第 3 章的管家看着它）→ 响应原路返回。这条链上的每一站，你现在都认识了。而 Vercel 呢？它把 443、证书、转发全部打包代劳了——平台的本质，就是替你雇好了这一整条链上的工作人员。用平台还是自己雇（DO + Nginx），是成本与控制的取舍，第 10 章见。

出事时想起我

502 Bad Gateway——前台在，后面的人死了（Nginx 活着，但 3000 号房没人应，去第 3 章 `systemctl status`）；Connection refused——那个端口根本没人在听（程序没起，或端口号对不上）；改了 Nginx 配置没反应——先 `nginx -t` 检查语法，再 reload，配置不会自己生效。

关键领悟

端口回答"找哪个房间"，反向代理回答"访客只需要知道大门"。443 门口那位 Nginx，就是把"一台机器上的很多程序"伪装成"一个体面网站"的全部秘密。

试一试（安全档：只读）

SSH 上 DO，跑 `ss -tlnp` 看看这台机器上谁在听哪些端口——每一行都是一个"房间与住户"。再找找 `/etc/nginx/sites-enabled/`：有，就把排班表逐行翻译给自己听；没有，也算重大发现——去问 AI："这台机器上的服务是怎么对外说话的？"

第 5 章 把表设计好：关系、外键、索引

第一册第 3 章说过：设计表结构，就是想清楚"产品到底要记住什么"。这一章给你三件设计工具，从此 AI 建的表你不光看得懂，还评得动。

工具一：一个事实只存一份。先看反面教材：如果 `certificates` 表里直接存一列 `owner_email`，会怎样？用户改一次邮箱，他名下一千张凭证就要改一千处——漏掉一张，数据从此打架，而且你永远不知道哪份是真的。正解你在第三册第 8 章已经见过：`certificates` 只存 `owner_id`，邮箱只住在 `users` 表里，一处修改，处处生效。这条朴素的原则是整个数据库设计的地基（教科书叫它规范化，记住人话版就够）。

工具二：外键，让数据库替你守规矩。`owner_id` 这个"工号"不只是约定——建表时声明出来，数据库会强制它有效。看一张真实形状的建表语句，AI 迁移文件里的那些话，现在全能读了：

📄sql

```
create table certificates (  
  id uuid primary key default gen_random_uuid(),  
  owner_id uuid references users(id) not null,  
  status text not null default 'draft',  
  created_at timestamptz default now()  
);
```

逐行读：`primary key` 是每行的身份证（`uuid` 是一种全球不重复的随机编号，默认自动生成）；`references users(id)` 就是外键——从此想塞一个不存在的 `owner_id`，数据库直接拒收，它从"记录事实"升级成了"守护事实"；`not null` 是"此列不许空着"；`default` 是没填时的默认值。关系的形态顺带认全：一对多（一个用户多张凭证——外键放"多"的那边），多对多（凭证和标签互相多——需要一张中间表牵线），一对一（少见）。

工具三：索引，书后的检索目录。没有索引时，`WHERE owner_id = ...` 等于逐页翻完整本书；跑一句 `create index on certificates(owner_id);` 之后，等于直接翻书后目录。代价是写入稍慢、占点空间，所以只给"经常用来找"的列建——经验法：外键列和高频出现在 `WHERE` 里的列（主键自带，不用操心）。第 8 章你会看到：九成"越用越慢"，最后都查到一条缺索引的查询头上。

三件工具合成一个习惯：以后让 AI 建表，需求里先让它回答三问——有哪些"东西"（表）？它们怎么相连（外键）？最常按什么找（索引）？三问入了需求，AI 的建表质量立刻上一个档。

出事时想起我

同一个信息在两处对不上——设计违反了"只存一份"；数据越多越慢——查缺失的索引；删用户时报 `violates foreign key constraint` ——那是数据库在守护事实：他名下还有凭证呢。别硬删，这属于"想清楚再动"的决策（级联删除是把双刃剑，动它之前叫人）。

关键领悟

好的表设计三句话——一个事实只存一份；关系交给外键，让数据库强制守护；常走的查找路径铺上索引。

试一试（安全档：只读）

打开 Supabase，把我们最重要的两三张表画成一张纸上关系图：方框是表，箭头是外键，箭头旁标上"一对多"。画不出的那根箭头，就是你下次和 AI 开会的第一个议题。

第 6 章 RLS 策略实战：亲手写"谁能看哪一行"

三次铺垫，今天兑现：第一册第 15 章立了规矩（没开 RLS 的表约等于公开），第三册第 8 章认了脸（`create policy` 是 SQL），这一章，亲手读写。

先懂为什么 RLS 非存在不可。传统架构里，前端永远隔着后端说话，权限由后端代码把守。而 Supabase 的架构允许前端带着用户的身份令牌直连数据库——快、省事，但把守者没了中间层可站，只能住进数据库本身。RLS（Row Level Security，行级安全）就是刻在每张表上的门规。

机制两句话说完：表一旦开启 RLS，默认拒绝一切；每条 policy 是一条"允许"——为某种操作、某种人，开一条缝。核心咒语只有一个：`auth.uid()` ——"现在敲门的这个人的用户编号"。看第一块标本：

📄sql

```
alter table certificates enable row level security;

create policy "owner can read own certificates"
on certificates for select
using ( auth.uid() = owner_id );
```

逐行读：给这张表挂上门规；立一条名字讲人话的规矩；只管 select（读）；条件——敲门人就是这一行的主人。`using` 读作"已存在的行，满足此条件才对你可见"。再看第二块标本，管"写入"的：

📄sql

```
create policy "user inserts own certificates"
on certificates for insert
with check ( auth.uid() = owner_id );
```

`with check` 读作"新来的行，必须满足此条件才收"——防止任何人替别人塞数据。这一对分清，RLS 就通了：`using` 管旧行给不给看、给不给改；`with check` 管新行收不收。顺带一种特殊写

法：`using (true)` 是"谁都能看"，用于确实要公开的数据（比如凭证的公开验证页）。写 `true` 没有错，但必须是故意的公开，不是忘了写条件。

然后是比较重要的事：测。`policy` 写完必须用两个身份测，而且要测满两面——自己能看到自己的，且看不到别人的。只测前半句不算测：权限的 `bug` 全部藏在"看不到"那半句里。Supabase 面板可以模拟不同身份，或者干脆开两个测试账号。最后把这套变成制度：以后让 AI 建任何表，需求里永远带一句——"开启 RLS，写明每条 `policy`，并用人话向我解释每条的含义。"第一册那句追问，今天正式入宪。

出事时想起我

前端查询永远返回空数组，可表里明明有数据——RLS 默认拒绝、缺 `policy`，本层最经典症状；用户 A 看到了 B 的数据——`policy` 条件写松了，按事故处理，立刻收紧并排查影响面；`insert` 报 `new row violates row-level security` ——`with check` 没过，检查新行的归属字段。

关键领悟

RLS 的世界里，默认是关死的门，`policy` 是开出的缝。`using` 管旧行，`with check` 管新行；每条缝都要用"看得到自己 + 看不到别人"两面测过才算存在。

试一试（安全档：△ 只在测试表上做）

和 AI 一起建一张练手表，亲手写上面那两条 `policy`，然后用两个账号完成两面测试。整套流程在测试表上走熟之前，不碰真表的 `policy`——这句话本身，就值得抄进红线卡。

第 7 章 自己开窗口：设计一个像样的 API

第一册第 4 章你学会了用别人的窗口；这一章学开自己的。这不是假设题——EchoForge 迟早（或已经）要把"验证一张凭证"这样的能力开放给别人的程序调用。窗口一旦开出去，设计的好坏就写在脸上了。好设计有四根柱子。

柱子一：名词做 URL，动词交给 HTTP。资源是名词复数：`/v1/certificates` 是凭证们，`/v1/certificates/:id` 是其中一张。动作用第一册学过的 HTTP 动词表达：GET 查、POST 建、PATCH 改、DELETE 删。URL 里冒出动词（`/getCertificate`、`/doVerify`）不致命，但一闻就不专业——像菜单上写"进行一个鱼的吃"。

柱子二：响应有统一的形状，失败也要设计。成功长什么样、失败长什么样，第一天就定下来，所有端点共用。失败的标本：

json

```
{ "error": { "code": "not_found", "message": "certificate not found" } }
```

配上状态码的几张头牌脸：200 成了；400 你的请求有毛病；401 你是谁；403 你不许；404 没这东西；500 我这边坏了。这些数字第 9 章读日志时天天见——统一的错误形状加准确的状态码，就是把排障从猜谜变成查表。

柱子三：钥匙进请求头，永不进 URL。调用方的 API Key 放在请求头里（`Authorization: Bearer` 一串钥匙）。为什么绝不放 URL？因为 URL 会进访问日志、进浏览器历史、进各种统计——把钥匙写在 URL 里，等于把家门钥匙写在明信片上寄出去。（第一册第 9 章的纪律，在窗口的另一侧同样成立。）

柱子四：版本号，从第一天开始。所有路径以 `/v1/` 开头。为什么第一个用户还没来就要版本？因为窗口一旦有人用，你的每次改动都会砸在他们头上。`/v1` 是一份冻结的承诺：这里的行为不再变，想改，开 `/v2`。听出来了吗——已发布的 API、已推送的 Git 历史、已部署的链上合约，是同一家人：进了公共世界的东西，只能追加，不能背叛。这是这套书第三次遇到这条定律，它值得三次。

最后半根柱子：文档即产品。窗口开了没人会用，等于没开。最小文档标准：每个端点一行——方法、路径、吃什么、吐什么，配一个能直接复制的调用例子。让 AI 生成初稿，你负责校对"它说的和窗口真实行为是否一致"。

出事时想起我

调用方说"你们 API 坏了"——先要三样东西：状态码几？完整请求长什么样？打的哪个版本？有了统一形状和版本号，这三问几乎总能当场定位；反过来，如果这三问问不出所以然，说明四根柱子有一根没立好。

关键领悟

API 设计的本质是替未来的调用者减少惊讶——包括未来的自己和 AI。名词做 URL、统一响应形状、钥匙进请求头、版本从第一天开始，四根柱子立住，窗口就体面了。

试一试（安全档：纸上作业）

为"验证一张凭证"设计一个只读端点：写下方法、路径、成功响应的形状、两种失败（不存在 / 未授权）的响应和状态码。写完先让 AI 挑刺，再拿本章四根柱子自查一遍——两轮下来还站得住的设计，就可以进需求文档了。

第 8 章 缓存与性能：快，是设计出来的

性能这门课的第一定律听起来像偷懒：最快的工作，是不做的工作。缓存（Cache）就是这条定律的化身——把算过的答案记下来，下次直接给。

其实你已经在这套系统的四层里分别见过它了，串起来看：浏览器缓存——静态文件存在访客本地，所以"强制刷新"（Ctrl+Shift+R）这个动作才存在；CDN 缓存——第 1 章 Cloudflare 把答案存在离访客近的节点；应用层缓存——程序把耗时的计算结果记在内存里（认个脸：Redis，一个专职记答案的内存数据库，见到不慌）；往宽了说，第 5 章的索引也是同族——都在用空间换时间。

缓存的代价只有一个，但很大：答案可能是旧的。TTL（第 1 章的老朋友）是答案的保质期；Purge / 失效，是主动把旧答案扔掉。行业里有句老话，说计算机科学只有两大难题，其中一个就是缓存失效——所以"我明明改了，怎么还是旧的"是全行业共享的痛。排障口诀：从近到远清——先强刷浏览器，再 Purge CDN，最后看应用层。

第二定律：先测量，再优化。凭感觉优化等于闭眼开枪。三个测量点从近到远：浏览器 F12 的 Network 面板（哪个请求最慢？）；Vercel 或服务器日志里的耗时；数据库的慢查询记录（Supabase 面板能看）。再给你一张粗糙但救命的量级直觉表：读内存是纳秒级，同机房查一次数据库是毫秒级，跨洋一个网络来回是上百毫秒级。看懂这张表就明白——慢，几乎从来不慢在你的循环和计算里，而是慢在网络往返的次数和没走索引的查询上。所以性能优化的前三板斧永远是：少跑几趟（把三个请求合成一个）、给查询铺索引（第 5 章伏笔回收）、给不变的东西上缓存。

最后一条平衡性的话：没有测量数据、没有用户抱怨之前，不要优化。过早优化偷走的是做功能的时间。我们的原则是——先让它对（第一册第 14 章），再让它稳（本册第 3、9 章），最后才让它快，而且只快该快的地方。

出事时想起我

"我改了怎么没生效"——缓存三连清，从浏览器到 CDN；"第一次打开慢、之后飞快"——冷缓存现象，正常；"越用越慢"——数据涨了，去抓缺索引的查询；"时快时慢"——多半是链条里某个外部 API 在抖，去日志里看耗时分布（下一章的活）。

关键领悟

快的秘诀不是更努力地算，而是少算（缓存）、少跑（合并往返）、走目录（索引）——而这一切开始之前，先测量。

试一试（安全档：只读）

F12 打开 Network 面板，刷新我们的网站，按耗时排序。回答三个问题：最慢的三个请求是谁？哪些标着"（来自缓存）"？最慢那一个，慢在哪一层——DNS、网络、还是服务器处理（点开能看到分段耗时）？把这个请求的故事讲给爸爸听。

第 9 章 日志、监控与告警：出事之前先知道

三个词先分家：日志是系统写的日记，出事后翻；监控是持续量体温的仪表盘，随时看；告警是体温超标自动打给你的电话，不用一直盯。大多数个人项目停在"有日志但不会看"这一级——今晚往上爬两级。

第一步：画出日志地图。我们的日志散落在四大件里，各有各的入口：Vercel 的函数日志（第一册第 14 章提过的那个）；Supabase 的数据库与 Auth 日志；DO 上，服务的日记用 `journalctl -u 服务名`（第 3 章的管家日记），Nginx 另有 access log 和 error log；Cloudflare 则有整体流量分析。于是排障的完整咒语终于凑齐了：症状定层（每章末尾攒的那张表），层定日志（本章这张地图）。两步走完，"网站坏了"就变成了"去翻某一本具体的日记"。

第二步：会读一行日志。拿 Nginx 的访问日志当标本：

□

```
203.0.113.7 -- [24/Jul/2026:20:14:03] "GET /api/v1/certificates/abc HTTP/1.1" 200 512 0.003
```

从左到右：谁（IP）、何时、要什么（方法加路径——第 7 章设计的窗口，在这里留下脚印）、结果如何（状态码，那几张头牌脸）、回了多大、花了多久。最原始也最有效的巡逻术，是第二册的 grep：`grep " 500 " access.log`——把所有"我这边坏了"的瞬间抓出来排队。

第三步：给 AI 立打日志的家规（可直接进 CLAUDE.md）：其一，关键动作必留痕——谁、何时、做了什么；其二，错误日志必须带上下文——只写一个 "failed" 是对未来自己的犯罪，要写"给谁、做什么、因为什么失败"；其三，秘密永不进日志——密码、完整的 API Key、令牌，一个都不许。日志天生会被到处传阅（贴给 AI、发给爸爸、存进监控平台），第一册第 9 章的纪律在这里长出了新的一面。

第四步：最小监控三件套。别贪多，三个数字起步：网站还活着吗——配一个 uptime 探针（这类免费服务每分钟替你打开一次网站，打不开就发邮件）；磁盘还剩多少——第 2 章说过，日志撑爆磁盘是裸服务器的头号杀手，让探针或定时任务盯着 `df`；错误率有没有突跳——各平台的仪表盘都有现成图。配告警时守一条黄金法则：每条告警必须对应一个行动。会响但不需要你做任何事的告警，唯一的作用是训练你忽略告警——那是给自己安装"狼来了"。宁少勿滥。

出事时想起我（本章是“想起我”们的总部）

用户说“昨晚打不开”而你毫不知情——缺 uptime 告警，今晚补上；排障时东翻西找像无头苍蝇——先默念咒语：症状定层，层定日志；日志翻到了却查不到关键信息——家规第二条没立，回去补上下文。

关键领悟

日志是写给“出事后的你”的信，监控是替你值班的眼睛，告警是只在需要行动时才响的铃。三件套的唯一目标：让你永远比用户先知道。

试一试（安全档：只读，外加一个小设置）

把四大件的日志入口各打开一次，做成一页“排障地图”（哪层的日志在哪看，一层一行）存进项目文档。然后给网站配上第一个 uptime 探针，告警发到你和爸爸都看得到的邮箱——这套书里第一位“替你值班”的员工，今晚上岗。

第 10 章 云账单怎么读：成本也是架构的一部分

云的付费模式是水电煤：用多少付多少。好处是起步几乎免费——我们能用四大件做出真东西，靠的就是这份慷慨。风险在另一头：水电煤没有上限，账单也没有。行业里最经典的恐怖故事，是一段死循环代码整夜狂调付费接口，或者流量被恶意刷爆，一觉醒来账单多了几个零。所以读账单不是抠门，是工程纪律的最后一块拼图。

先看懂三种计费模型，正好对应四大件：固定月租——DO 的服务器，像房租，用不用都是这个数。这是"自己养服务器"藏得最深的优点：账单天然封顶。用量计费——Vercel 的函数调用与流量、Supabase 的存储与带宽，像水电，随用户增长。免费额度加超量——四大件全都送慷慨的 free tier，这是个人项目常年零元的原因；但每一家都要搞清两件事：额度线画在哪？过线之后是"停"还是"扣"？有的服务超额直接暂停（安全但会宕机），有的默默开始计费（不宕机但花钱）——这个开关在各家的账单设置里，值得亲手确认一遍。

再养成三个习惯。其一，每月看一次账单，像体检——重点从来不是数字大小，而是"和上月比有没有异动"。异动必有故事：流量涨了？被爬了？哪段代码在狂奔？换句话说，账单是用钱写成的另一份监控日志——上一章的思路在这里完全适用，而且各平台都有支出提醒（budget alert），全部设上，让钱包也有自己的告警。其二，给会花钱的东西设限：付费 API 的 key 设配额，平台设支出上限。其三，定期清理僵尸：试验完忘了删的项目、闲置的服务器、自动续费的域名——云上最常见的浪费不是用太多，是忘了它还在扣钱。

最后，把成本请进架构决策。第一册第 5 章"代码住哪"的选择题，现在补上最后一个维度：偶尔醒的活，serverless 按次计费最便宜；永远醒且一直忙的活，固定月租的服务器反而省；大流量的静态内容，让 Cloudflare 的缓存去扛——免费的 CDN 流量是它送你的最大一份礼物。从此评估任何方案时多问一句："它的账单是什么形状的？"——问出这一句，你就是在做架构，不只是在写功能。（顺带一提这道题的另一个极端解：ForgeCard 那台树莓派，一次性硬件加一点电费，零月租——"自己家机房"的极致形态。没有最好的形状，只有合适的形状。）

出事时想起我

账单突然跳——先看用量图里哪条线异动（流量？调用次数？存储？），再对着异动的时间点去翻第 9 章的日志。成本异动，几乎永远是系统行为异动投在钱包上的影子。

关键领悟

账单不是财务问题，是用钱写成的监控。会读它，你既守住了钱包，又多了一双发现系统异常的眼睛。

试一试（安全档：只读加设置）

和爸爸开一次二十分钟的"账单会"，四大件的账单页各过一遍，回答三个问题：现在每月总成本是多少？哪一项离免费额度线最近？过线之后，各家是停还是扣？最后把每家的支出提醒全部设上。这个会，以后每月开一次。

写在第四册末尾

引擎盖可以合上了。盘点一下你现在身上多出来的东西：一张"症状 → 层 → 日志"的排障地图——这是本册真正的毕业证；一套评估表结构、权限、接口的标尺；还有一种新的目光——看任何方案时，会顺口问出"它的账单是什么形状""它出事时症状会是什么"。四大件从供应商，变成了你叫得出脾气的同事。

只剩最后一册了。工具会了，语言通了，系统懂了——第五册回答最后一个问题：怎么把人的判断和 AI 的马力，装进同一套制度里，让我们能又快又稳地，造更大的东西。

附：本册新词

- 源站 (Origin)：门面 (Cloudflare) 背后真正干活的机器。
- 橙云 (Proxied)：Cloudflare 的核心开关——开了，一切流量先过它的手。
- Purge：主动清空 CDN 里的旧答案。
- SSH / 密钥对：加密的远程终端 / 门上的锁 (公钥) 与兜里的钥匙 (私钥，永不外传，包括对 AI)。
- 指纹 (Fingerprint)：首次连接时记下的服务器"长相"，防冒充。
- 进程 (Process) / PID：活过来的程序 / 它的编号。
- systemd / unit / systemctl / journalctl：服务管家 / 登记表 / 指挥口令 / 管家日记。
- 端口 (Port)：IP 是楼，端口是房间；80、443、22 三大名门。
- 反向代理 (Reverse Proxy) / Nginx：替服务器在大门口接客的前台。
- 502 Bad Gateway：前台在，后面的人死了。
- 主键 / 外键：行的身份证 / 表间的工号——且由数据库强制守护。
- 索引 (Index)：书后的检索目录，用空间换时间。
- RLS / policy / using / with check：行级门规 / 开出的缝 / 管旧行 / 管新行。
- auth.uid()："现在敲门的这个人是谁"。
- Bearer / Authorization 头：钥匙的唯一正确携带位置——永不进 URL。
- /v1：对调用者的冻结承诺；改动进 /v2。
- 缓存 (Cache) / TTL / 失效：记下的答案 / 保质期 / 扔掉旧答案。
- Redis：专职记答案的内存数据库，认脸即可。
- uptime 探针：每分钟替你打开一次网站的值班眼睛。
- 告警 (Alert)：只在需要行动时才响的铃。
- free tier / 支出提醒：免费额度 / 钱包的烟雾报警器。

第四册《深入系统》v1.0 · 完 下一册：《AI 工程心法》——把人的判断和 AI 的马力，装进同一套制度。